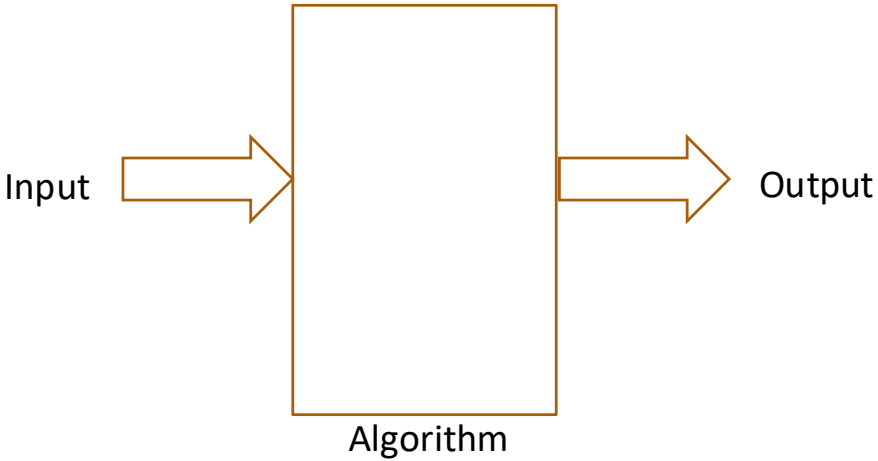
Randomized Algorithms

Randomized algorithms

- A randomized algorithm is just one that depends on random numbers for its operation, behavior depends on:
  - an input
  - a sequence of random bits
- Use of probability in the analysis of algorithms
- Can be faster and simpler than a deterministic (no random bits) algorithm

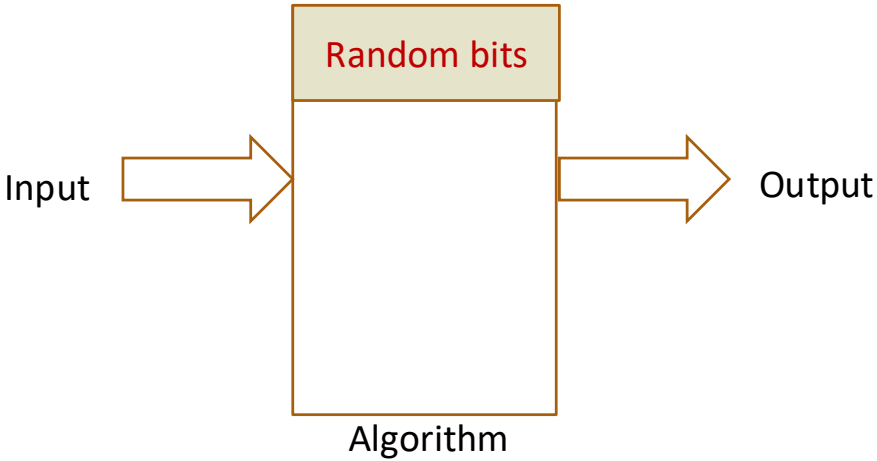
Deterministic Algorithm

A deterministic algorithm is an algorithm which, given a particular input, will **always produce the same output**, with the underlying machine always passing through the same sequence of states.



The **output** as well as the **running time** are **functions** only of the **input**.

Randomized Algorithm



The **output** or the **running time** are **functions** of the **input** and **random bits chosen** .

"Randomized algorithm for a problem is usually **simpler** and **more efficient** than its deterministic counterpart."

☆ The difference is important / could be in a test

Types of Randomized algorithms





Las Vegas (randomness is in execution time, guarantee correct output)

Monte Carlo (execution is guarantee but output won't always be correct)

Types of Randomized Algorithms

- **Randomized Las Vegas Algorithms:**
  - "everyone wins in Vegas"
  - Guaranteed to produce correct answer
  - Running time is probabilistic (a random variable)
    - running time fast with a high probability
  - Example: Randomized Quick Sort
- **Randomized Monte Carlo Algorithms:**
  - Output may be incorrect with some probability (returns correct answer with a high probability)
    - We can bound its probability, By running it many times on independent random variables, we can make the failure probability arbitrarily small *at the expense of running time*.
  - Running time is deterministic.
  - Example: Randomized algorithm for approximate median

Example 1: Quicksort

Choosing a random number as a pivot won't guarantee the sort time, could be nlogn or n^2

```
QuickSort(A,l,r)
{     If (l < r)
            x ← A[l];
          i ← Partition(A,l,r,x);
           QuickSort(A,l, i − 1);
           QuickSort(A,i + 1, r)
}
```

- Best case running time: O(n log n)
- Worst case running time: $O(n^2)$
- Distribution sensitive: time taken depends upon the initial permutation of A

Example 1: Randomized Quicksort

```
QuickSort(A,l, r)
{     If (l < r)
          x← an element selected randomly uniformly from A[l..r];
          i← Partition(A,l,r,x);
          QuickSort(A,l, i − 1);
          QuickSort(A,i + 1, r)
}
```

a count can be added to find the number of times it was randomized (after x gets randomized)

- Distribution sensitive: time taken does not depend on the initial permutation of A
- Time taken depends upon the random choice of pivot which always leads to a running time of **O(*n* log*n*)** in the average case

Example 2

- Suppose we want to find a number among n given numbers which is larger than or equal to the median.

  Suppose $A_1 < \ldots < A_n$.

  We want A_i, such that $i \geq n/2$.

- Deterministic algorithm needs O(n) time to produce the answer.
- Suppose n is 100,000,000,000 ! very large!

Choose the max of 100 numbers out of 10" for O(1) constant

Median

The median of a dataset is calculated by following two simple steps.

First, arrange the given data in ascending order.

Next, we need to pick the middlemost data.

For an even number of data points, there are two middle values, and we need to take the average of those two middle values.

Median example

1, 3, 3, **6**, 7, 8, 9

Median = **6**

1, 2, 3, **4**, **5**, 6, 8, 9

Median = (4 + 5) ÷ 2

= **4.5**

Example 2: Using Monte Carlo

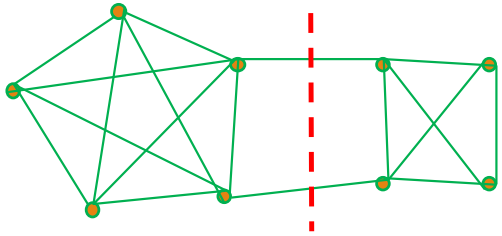
- Solution:
  - Choose 100 elements with *equal probability*.
  - Find the maximum among these numbers.
  - Return the maximum.
- The running time of the given algorithm is constant O(1).
- Consider that the algorithm may return a wrong answer, but the probability is very smaller than the hardware failure

To decrease the probability of failure, repeat the program multiple times

Example 3: Min-Cut

Problem definition: Given a connected graph **G**=(**V**,**E**) on **n** vertices and **m** edges, compute the smallest set of edges that will make G disconnected.

- Input: A connected, undirected graph G = (V,E)
- Output: A min-cut, i.e. smallest set of edges to remove that splits graph into two



Possible approach:

- Try them all – exponential since there are $2^{|E|}$ subsets of edges

Brute force

Example 3: Min-Cut

- **Best deterministic algorithm : [Stoer and Wagner, 1997]**
  - **O(mn)** time complexity.
- **Randomized Monte Carlo algorithm: [Karger, 1993]**
  - **O(m log n)** time complexity.
  - Error probability: $\mathbf{n}^{-c}$ **for any** c **that we desire**

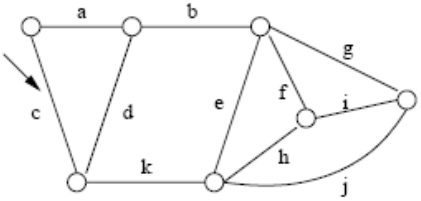
Randomized idea:

- one time through the algorithm yields a relatively low probability of success
- repeat algorithm to reduce probability of repeated failure until satisfied with probability of success

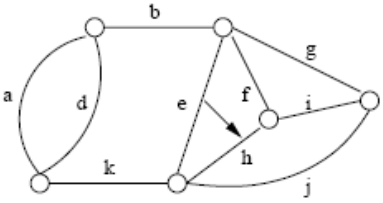
Example 3: Min-Cut

Algorithm:

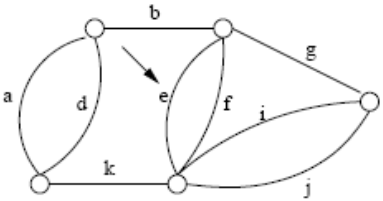
1. Randomly select an edge (u,v)
2. Merge its end-points u and v
3. Repeat until two vertices remain
4. Return the cut



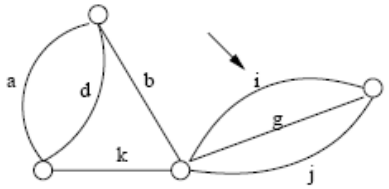
Step 1



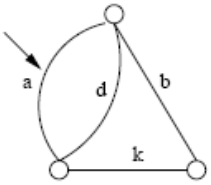
Step 2



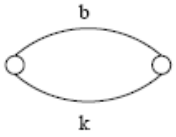
Step 3



Step 4



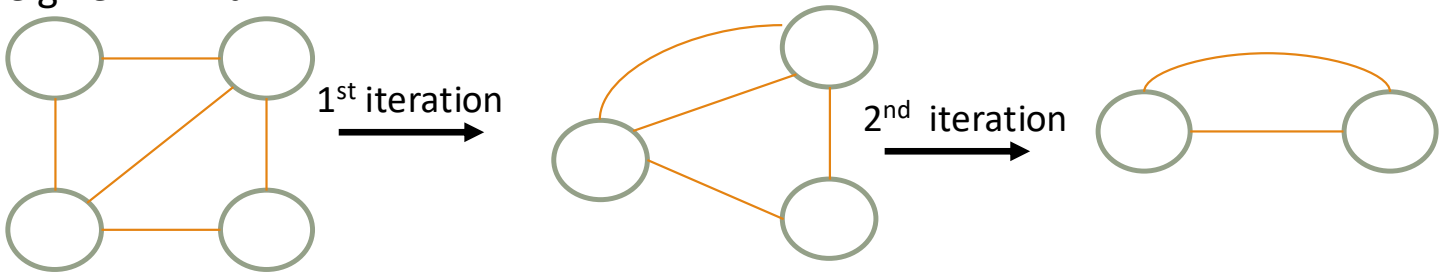
Step 5



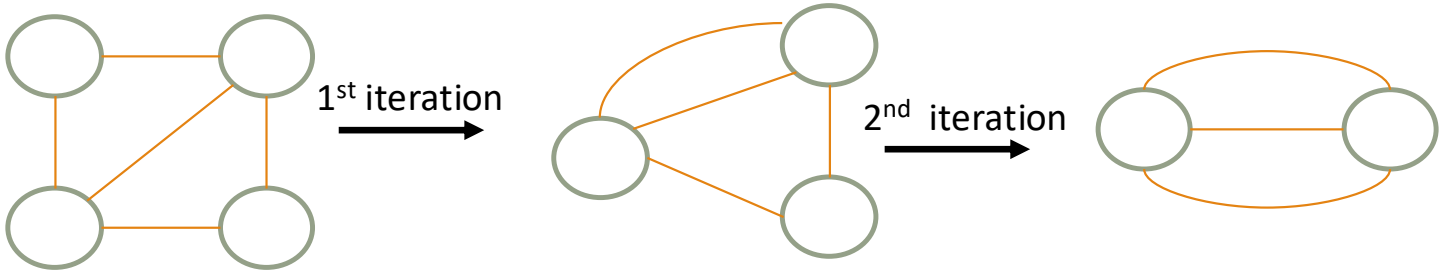
Step 6

Example 3: Min-Cut

Grave given: 1st run



Same Grave: 2nd run



The algorithm does not always produce the correct result. The run shown in the top correctly determines the minimum cut size = **2**. The run shown in the bottom fails to produce the correct result; the algorithm will claim the size of the minimum cut is **3**.

Example 4: Primality Test

- Goal: determine if a number n is prime
- Output: YES or NO
- Useful in cryptography
- Brute force solutions:
  - O(n) or it could be optimized to check till sqrt(n)

divide the number by every number less than it

ex: 100
$\sqrt{100} = 10$ so divide by 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Example 4: Primality Test

- Fermat's Little Theorem: (could fail for some numbers)
  - If n is prime, then for all a in [1,n-1],
    - $a^n - a = 0 \pmod{n}$

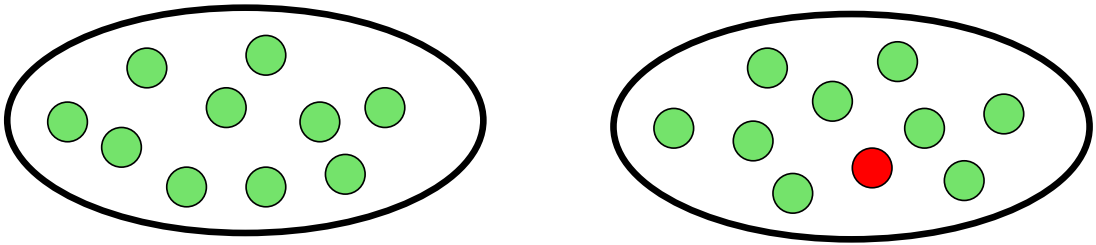
First attempt at algorithm:

1. Choose a at random from [1,n-1]
2. If $a^n - a = 0 \pmod{n}$ fails, then return COMPOSITE
3. Else return PRIME

Example: Microchip Test(Read it)

We want to know if the manufacturer tested a batch of chips

For an untested batch, P(bad) = 0.1



Deterministic algorithm: test them all => O(n)

Randomized algorithm: test chips at random

Example: Microchip Test

Question to ask: “Has this batch NOT been tested?”

- **TRUE** – if random algorithm finds a bad chip
- **UNKNOWN** – if chip finds a good chip

After *k* chips, algorithm concludes FALSE, therefore the batch is GOOD.

Probability a good chip came from untested batch = 0.9

Probability “good” batch is actually bad = 0.9^k

Example 6: Microchip Test

0.9^k = probability a bad batch is being tested and we keep finding the good chips, k times in a row (how many times we tested or has been repeated)

For k = 66, chances of a bad batch slipping through is less than 1 in a 1000 => **small**

Running time = O(k), independent of n, determined by k set by programmer => **fast**

Monte Carlo: running time fast, correctness probability high

Pseudorandom numbers

The computer is *not capable* of generating truly random numbers as theres a formula

- The computer can only generate pseudorandom numbers--numbers that are generated by a formula
- Pseudorandom numbers *look* random, but are perfectly predictable if you know the formula
  - Pseudorandom numbers are good enough for most purposes, but not all--for example, not for serious security applications
- Devices for generating truly random numbers do exist
  - They are based on radioactive decay, or on lava lamps

"Anyone who attempts to generate random numbers by deterministic means is, of course, living in a state of sin." —John von Neumann

Generating random numbers

Perhaps the best way of generating "random" numbers is by the linear congruential method (LCG):

- $X_{n+1} = (a * X_n + b) \% m;$
  where a and b are large prime numbers, and m is 2^{32} or 2^{64}
- The initial value of X_0 is called the seed
- If you start over with the same seed, you get the same sequence of "random" numbers

One advantage of the linear congruential method is that it will (eventually) cycle through all possible numbers

Getting (pseudo)random numbers in Java

1. import java.util.Random;
2. new Random(long seed) // constructor

   new Random() // constructor, uses System.timeInMillis() as seed
3. To create a pseudo-random number, use one of the following functions
   - nextBoolean()
   - nextFloat(), nextDouble() // $0.0 \leq$ return value < 1.0
   - nextInt(), nextLong() // all 2^{32} or 2^{64} possibilities
   - nextInt(int n) // $0 \leq$ return value $< n$
   - nextGaussian()
     - Returns a double, normally distributed with a mean of 0.0 and a standard deviation of 1.0
4. void setSeed(long seed) //reset the generator

Randomized Algorithms: Recap

Faster and simpler: Even when a fast deterministic algorithm exists, it is more complex

Lots of theory usually needed in the specific application area to prove either:

- a reasonable one-time successful probability, such that repeating would lead to good overall probability (***Monte Carlo***)
- a reasonable expected running time (***Las Vegas***)

Just as good: High probability of success just as good as deterministic success: P = 0.5^{100} is smaller than probability of your hard drive being hit by a meteor…while running the algorithm

Conclusions

Sometimes you want random numbers for their own sake—for example, you want to shuffle a virtual deck of cards to play a card game

Sometimes you can use random numbers in a simulation (such as the queue example) to avoid difficult mathematical problems, or when there is no known feasible algorithm (such as the 0-1 knapsack problem)

Randomized algorithms are basically *experimental*—you almost certainly won't get perfect or optimal results, but you can get "pretty good" results

Typically, the longer you allow a randomized algorithm to run, the better your results

A randomized algorithm is what you do when you don't know what else to do

- As such, it should be in every programmer's toolbox!

Sources/References/Further Reading

Chu, J., & Amstutz, P. (2002). Scribe notes for Lecture 12, U.Mass CMPSCI611. *Advanced Algorithms*.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2001). *Introduction to algorithms*. Cambridge, Mass.: MIT Press.

Edmonds, J. (2005). How to think about algorithms.

Goodrich, M.T., & Tamassia, R. (2002). *Algorithm Design*. New Jersey, NY: John Wiley & Sons, Inc.

http://mathworld.wolfram.com

Motwani, R., & Raghavan, P. (1995). *Randomized algorithms*. New York, NY: Cambridge University Press.

Rosen, K. H. (2000). *Elementary number theory and its applications*. Reading, Mass.: Addison Wesley Longman.

Rosen, K. H. (2003). *Discrete mathematics and its applications*. Boston: McGraw-Hill.